PROGRAM 1: DOMESTIC ANIMALS

Aim

To write a Java program demonstrating default and parameterized constructors and method overloading using domestic animals.

Description

The DomesticAnimal class stores an animal's name, shelter, and diet. It has a default constructor and a parameterized constructor. The showDetails() method displays animal information. The makeSound() method is overloaded: one version displays a sound, while the other displays a sound repeated a specified number of times.

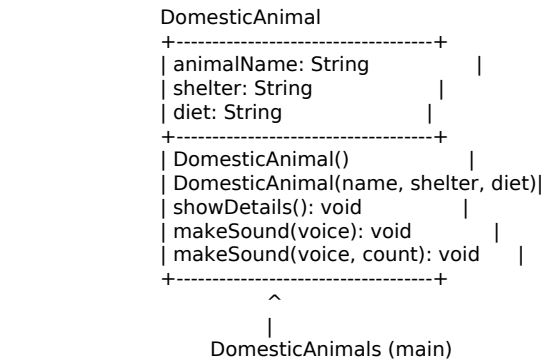
Software Requirements

Hardware: Computer or laptop. Software: Java Development Kit (JDK) and a Java IDE or text editor.

Algorithm

1. Start the program.
2. Define DomesticAnimal with animalName, shelter, and diet fields.
3. Define a default constructor and a parameterized constructor.
4. Define showDetails() to print animal details.
5. Overload makeSound() with one parameter and with two parameters.
6. Create Goat, Sheep, and Dog objects in main().
7. Display each animal's details and call the appropriate sound method.
8. Stop the program.

UML Diagram



Code

```
package animalprog;

class DomesticAnimal {

    String animalName;
    String shelter;
    String diet;

    // Default constructor
    DomesticAnimal() {
        animalName = "Unknown";
        shelter = "Farm";
        diet = "Food";
    }

    // Parameterized constructor
    DomesticAnimal(String animalName, String shelter, String diet) {
        this.animalName = animalName;
        this.shelter = shelter;
        this.diet = diet;
    }

    void showDetails() {
        System.out.println("Animal: " + animalName);
        System.out.println("Shelter: " + shelter);
```

```
        System.out.println("Food: " + diet);
    }

    void makeSound(String voice) {
        System.out.println("Sound: " + voice);
    }

    // Method overloading
    void makeSound(String voice, int count) {
        System.out.println("Sound: " + voice);
        System.out.println("Repeated: " + count + " times");
    }
}

public class DomesticAnimals {

    public static void main(String[] args) {

        DomesticAnimal goat = new DomesticAnimal(
                "Goat", "Goat Shed", "Leaves");

        DomesticAnimal sheep = new DomesticAnimal(
                "Sheep", "Sheep Pen", "Grass");

        DomesticAnimal dog = new DomesticAnimal(
                "Dog", "Kennel", "Dog Food");

        goat.showDetails();
        goat.makeSound("Bleat");
        System.out.println();

        sheep.showDetails();
        sheep.makeSound("Baa");
        System.out.println();

        dog.showDetails();
        dog.makeSound("Bark", 3);
    }
}
```

Output Screenshot

```
Animal: Goat
Shelter: Goat Shed
Food: Leaves
Sound: Bleat

Animal: Sheep
Shelter: Sheep Pen
Food: Grass
Sound: Baa

Animal: Dog
Shelter: Kennel
Food: Dog Food
Sound: Bark
Repeated: 3 times
```

Result

Thus, the Java program demonstrating constructors and method overloading with domestic animals was written and executed successfully.

PROGRAM 2: ORDER DETAILS

Aim

To write a Java program demonstrating inheritance by managing client details and urgent and regular orders.

Description

The Client class stores the customer's name and city and provides methods to place and collect orders. The Purchase class stores order details and provides approval and completion methods. UrgentOrder and RegularOrder inherit from Purchase and define their own send() methods. RegularOrder also has an accept() method.

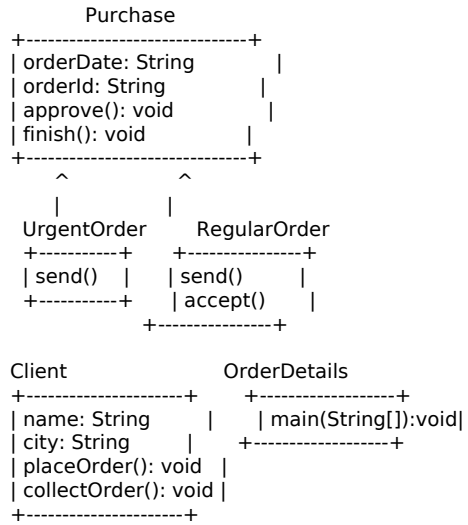
Software Requirements

Hardware: Computer or laptop. Software: Java Development Kit (JDK) and a Java IDE or text editor.

Algorithm

1. Start the program.
2. Create a Client object and assign the customer name and city.
3. Create UrgentOrder and RegularOrder objects.
4. Assign the date and ID to each order.
5. Display customer details and call placeOrder().
6. Display urgent-order details and call approve(), finish(), and send().
7. Display regular-order details and call approve(), finish(), send(), and accept().
8. Call collectOrder() and stop the program.

UML Diagram



Code

```
package ordermanagement;

class Client {

    String name;
    String city;

    void placeOrder() {
        System.out.println(name + " placed the order.");
    }

    void collectOrder() {
        System.out.println(name + " collected the order.");
    }
}

class Purchase {
    String orderDate;
```

```
    String orderId;

    void approve() {
        System.out.println("Order approved.");
    }

    void finish() {
        System.out.println("Order completed.");
    }
}

class UrgentOrder extends Purchase {

    void send() {
        System.out.println("Urgent order sent.");
    }
}

class RegularOrder extends Purchase {

    void send() {
        System.out.println("Regular order sent.");
    }

    void accept() {
        System.out.println("Regular order accepted.");
    }
}

public class OrderDetails {

    public static void main(String[] args) {

        Client customer = new Client();
        customer.name = "Anjali";
        customer.city = "Hyderabad";

        UrgentOrder urgent = new UrgentOrder();
        urgent.orderDate = "15-08-2026";
        urgent.orderId = "U201";

        RegularOrder regular = new RegularOrder();
        regular.orderDate = "15-08-2026";
        regular.orderId = "R202";

        System.out.println("Customer: " + customer.name);
        System.out.println("City: " + customer.city);
        customer.placeOrder();

        System.out.println("\nUrgent Order");
        System.out.println("Date: " + urgent.orderDate);
        System.out.println("Order ID: " + urgent.orderId);
        urgent.approve();
        urgent.finish();
        urgent.send();

        System.out.println("\nRegular Order");
        System.out.println("Date: " + regular.orderDate);
        System.out.println("Order ID: " + regular.orderId);
        regular.approve();
        regular.finish();
        regular.send();
        regular.accept();

        customer.collectOrder();
    }
}
```

Output Screenshot

```
Customer: Anjali
City: Hyderabad
Anjali placed the order.

Urgent Order
Date: 15-08-2026
Order ID: U201
Order approved.
Order completed.
Urgent order sent.

Regular Order
Date: 15-08-2026
Order ID: R202
Order approved.
Order completed.
Regular order sent.
Regular order accepted.
Anjali collected the order.
```

Result

Thus, the Java program demonstrating inheritance for urgent and regular order processing was written and executed successfully.